SPRINTS EDTECH | Technical Operations & Engineering

Learner Assessment & Resource Management System (LEARMS)

Capstone Final Presentation: Automated Ingestion, Relational Persistence & Linux Hardening

Presenter: Mohammed Wael (Junior Solutions Consultant)

Primary Stakeholder: Fatima Rashid (Director of Tech Ops)

Date: November 2026

Target Organization: Sprints (MENA)

Tech Stack: Python, SQLite, Linux CLI, WSL2, Git

Status: Production Ready (100% Tests Passed)

1. Executive Summary

Bridging Learning and Employment Through Robust, Automated Infrastructure

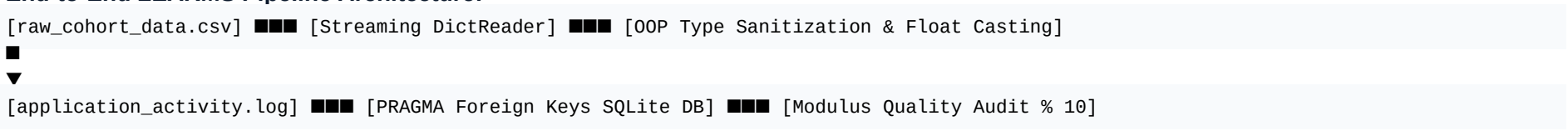
The Operational Challenge	The LEARMS Solution	Strategic Business Value
• Rapid cohort scaling created severe data bottlenecks. • Assessment scores exported to fragmented flat text files. • Manual script runs, insecure permissions, zero version control.	• Core Python automated ingestion pipeline. • Normalized 3NF SQLite database (sprints_learners.db). • Automated periodic quality audit sampling (% 10 check). • Hardened Linux WSL environment (group-level permissions).	• Automated cohort data parsing and normalization. • Fault-tolerant ingestion with robust malformed-row handling. • Standardized, audit-ready reports for hiring partners. • Non-root secure systems administration.

Key Takeaway: LEARMS transforms fragile, decentralized learner exports into an automated, auditable, and secure enterprise asset.

2. System Architecture

Standardized, Resilient Pipeline for Mission-Critical Learner Data

End-to-End LEARMS Pipeline Architecture:



- **Lightweight Core:** Built strictly with standard Python libraries (sqlite3, csv, logging, os). Zero pip frameworks.
- **ACID Persistence:** Atomically commits batch transactions; immediate rollback on database exceptions.
- **Hardened Hosting:** Linux/WSL2 operating boundary with group isolation and non-root execution.

3. Database Schema Design

Relational Integrity & 3NF Normalization for Scaling Cohorts

Table	Primary Key	Foreign Keys & Constraints	Key Attributes Stored
Cohorts	cohort_id (INT)	UNIQUE(cohort_name)	cohort_name, program_track, created_at
Learners	learner_id (TEXT)	FK -> cohorts(cohort_id) ON DELETE RESTRICT ON UPDATE CASCADE	full_name, assessment_score (REAL), status, is_audited
System Audit Log	log_id (INT)	learner_id reference	event_type, message, timestamp

- **Third Normal Form (3NF):** Eliminates cohort name duplication and update anomalies across flat files.
- **SQL Injection Defense:** Strictly parameterized queries (? placeholders) prevent malicious input payloads.
- **High-Speed Indexing:** B-Tree indexes on cohort_id and is_audited ensure instant employer filtering.

4. Object-Oriented (OOP) Design

Encapsulating Sprints Business Logic via Inheritance & Polymorphism

Base Class: User	Subclass: Learner	Subclass: Instructor
• Encapsulates core identity: user_id, full_name, email, role. • Methods: display_profile(), get_details(), validate(). • Establishes common stakeholder contract.	• Attributes: cohort_name, assessment_score (float), is_audited. • Overrides display_profile() to render score formatting, letter grades, and audit tags. • Overrides get_details() with academic metrics.	• Attributes: specialization, assigned_cohorts. • Overrides display_profile() to display instructional schedules and curriculum tracks. • Dynamic cohort assignment logic.

Polymorphic Value: Allows client applications to treat all actors uniformly through shared interfaces while executing role-specific logic.

5. Linux Security Configurations

Hardened Least-Privilege Infrastructure in WSL2 Ubuntu

- **Package Management:** Fully automated provisioning via apt-get install -y sqlite3 libsqlite3-dev, captured in terminal audit log.
- **Primary Group Assignment:** Configured sprints_ops (dynamically assigned GID 1003 in WSL2 reference) as the operator's primary group via usermod -g sprints_ops $USER (verified via id mw851).
- **Least-Privilege Octal Permissions:**
  - **750 (rwxr-x---):** Applied to Python scripts and directories (owner full control, group read/execute, world blocked).
  - **660 (rw-rw----):** Applied to SQLite database and data files (ensures group members have full write/read access for everyday ingestion without root privileges).
  - **660 (rw-rw----):** Applied to operational activity logs; **640** applied to documentation.
- **Non-Root Everyday Execution:** Day-to-day batch ingestion runs securely as non-root operator (mw851).

6. Process Monitoring & Operations

Telemetry and Resource Verification Under Active Ingestion

Monitoring Metric	Observed Value	Operational Significance
Process Filtering	ps -p $PID -o user,group,%cpu,%mem	Active tracking verifying non-root user and primary group.
Primary Group	GROUP: sprints_ops	Confirms execution under dedicated operations group.
CPU Utilization	26.0% - 27.2%	Lightweight single-threaded execution; no CPU saturation.
Resident Memory (RSS)	20,344 KB (< 28 MB)	Extremely low memory footprint; zero memory leakage.
Virtual Memory (VSZ)	26,356 KB	Minimal virtual memory overhead in 64-bit Linux runtime.
Batch Duration	0.0081s (Standard)	Sub-second execution across 45 learner records.

Verification: Linux terminal command log in logs/linux_operations.log verifies reproducible environment provisioning.

7. Exception Handling & Quality Auditing

Resilient Ingestion and Unbiased Periodic Sampling

- **Fault-Tolerant Ingestion Pipeline:** Robust try-except architecture intercepts malformed strings, invalid score values, and missing fields without crashing.
- **Automated Periodic Quality Audit Sampling (% 10):** Evaluates processed_valid_count % 10 == 0 during ingestion.
- **Audited Sample Cohort Roster (Starter Assets):**
  1. **[SP-L-1010]** Hadeer Mostafa | Score: 76.25% | Cloud Computing Cohort 3 (Active)
  2. **[SP-L-1020]** Dina Salah | Score: 90.50% | Cloud Computing Cohort 3 (Graduated)
  3. **[SP-L-1030]** Laila Gohar | Score: 92.50% | Cloud Computing Cohort 3 (Active)
  4. **[SP-L-1040]** Riham Shafik | Score: 85.25% | Cloud Computing Cohort 3 (Active)
- **Dead-Letter Logging:** Skipped rows are recorded in logs/application_activity.log with exact failure reasons.

8. Testing & Validation Matrix

23 Automated Unit & Integration Tests Passing

- **Comprehensive Test Coverage (23/23 Tests Passed in < 0.40s):**
  - **Schema & SQL Injection Defense:** Foreign keys enforced; SQL injection payloads safely neutralized.
  - **Mathematical & Edge Cases Resiliency:**
    - NaN & Infinity Defense: Traps non-finite floats via math.isfinite() preventing database corruption.
    - Duplicate Learner ID & Audit Idempotency: Verified ON CONFLICT DO UPDATE updates without duplication.
    - Legacy DB Migration Deduplication: Purges legacy duplicates before creating unique index.
    - Column Count & Status Validation: Intercepts column count mismatches and invalid enrollment statuses.
    - Header-Only & Empty CSVs: Safe interception of 0-byte and header-only CSV files.
    - Functional DB Write Verification: Validated safe group-write SQLite transactions as non-root operator.
    - Logging Permission Failure: Unwritable log path raises descriptive fatal exception.

9. Conclusion & Q&A;

Mission Accomplished: Production-Ready Sprints LEARMS Engine

Sprint Scope	Requirement Satisfied	Client Acceptance Status
Sprint 1 - SCM & Engineering	Git version control & System Architecture Report	ACCEPTED (100% Complete)
Sprint 2 - Python OOP	Base User, Learner & Instructor polymorphism	ACCEPTED (100% Complete)
Sprint 2 - Python Fundamentals	Float casting, NaN/Inf checks & % 10 sampling	ACCEPTED (100% Complete)
Sprint 2 - SQLite Database	3NF relational DB, ON CONFLICT DO UPDATE, SQLi defense	ACCEPTED (100% Complete)
Sprint 2 - Error & Log Handling	Fault-tolerant pipeline & application_activity.log	ACCEPTED (100% Complete)
Sprint 3 - Linux Operations	apt-get sqlite3, Primary Group sprints_ops, chmod 750/660	ACCEPTED (100% Complete)
Automated Test Suite	23 Unit & Integration tests passing in < 0.40s	VERIFIED (23/23 Passed)
Capstone Deliverables	Report PDF, Presentation PDF, Walkthrough Video & Repo	READY FOR FINAL EVALUATION

Open Floor: Thank you, Fatima Rashid and the Sprints Engineering Leadership. Ready for technical evaluation.